100+ Tricky Go Guess-the-Output Questions (with Explanations)

Curated from common interview patterns seen across Go interview prep resources and MCQ-style question banks.

Section 1: Basics, Declarations, and Scope (Q1-12)

Q1

```go
package main
import "fmt"

func main() {
    var x int
    fmt.Println(x)
}
```

Answer: `0` **Explanation:** Zero value of `int` is `0`.

Q2

```go
package main
import "fmt"

func main() {
    x := 10
    {
        x := 20
        fmt.Println(x)
    }
    fmt.Println(x)
}
```

Answer: `20` then `10` **Explanation:** Inner `x` shadows outer `x`.

Q3

```go
package main
import "fmt"

var x = 1

func main() {
    x := x + 1
    fmt.Println(x)
}
```

Answer: `2` **Explanation:** Right-hand side uses package-level `x`.

Q4

```
package main
import "fmt"

func main() {
    var a, b = 2, 3
    a, b = b, a
    fmt.Println(a, b)
}
```

Answer: `3 2` **Explanation:** Parallel assignment swaps values.

Q5

```
package main
import "fmt"

func main() {
    const x = 7
    fmt.Println(x, x+1)
}
```

Answer: `7 8`

Q6

```
package main
import "fmt"

func main() {
    arr := [...]int{1, 2, 3}
    fmt.Println(len(arr), cap(arr))
}
```

Answer: `3 3`

Q7

```
package main
import "fmt"

func main() {
    s := "go"
    fmt.Println(len(s))
}
```

Answer: `2`

Q8

```
package main
import "fmt"

func main() {
    var s string
    fmt.Println("[" + s + "]")
}
```

Answer: `[]` **Explanation:** Zero value of string is empty string.

Q9

```
package main
import "fmt"

func main() {
    var p *int
    fmt.Println(p == nil)
}
```

Answer: `true`

Q10

```
package main
import "fmt"

func main() {
    m := map[string]int{"a": 1}
    v, ok := m["b"]
    fmt.Println(v, ok)
}
```

Answer: `0 false`

Q11

```
package main
import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        defer fmt.Print(i)
```

```
    }
}
```

Answer: `210` **Explanation:** `defer` is LIFO.

Q12

```
package main
import "fmt"

func main() {
    defer fmt.Println("A")
    fmt.Println("B")
}
```

Answer: `B` then `A`

Section 2: Slices, Arrays, and Maps (Q13-24)

Q13

```
package main
import "fmt"

func main() {
    a := [3]int{1, 2, 3}
    b := a
    b[0] = 99
    fmt.Println(a[0], b[0])
}
```

Answer: `1 99` **Explanation:** Arrays are copied by value.

Q14

```
package main
import "fmt"

func main() {
    a := []int{1, 2, 3}
    b := a
    b[0] = 99
    fmt.Println(a[0], b[0])
}
```

Answer: `99 99` **Explanation:** Slices share backing array.

Q15

```go
package main
import "fmt"

func main() {
    s := []int{1, 2, 3, 4}
    t := s[1:3]
    fmt.Println(len(t), cap(t))
}
```

Answer: `2 3`

Q16

```go
package main
import "fmt"

func main() {
    s := []int{1, 2}
    s = append(s, 3)
    fmt.Println(s)
}
```

Answer: `[1 2 3]`

Q17

```go
package main
import "fmt"

func main() {
    m := map[string]int{"x": 1}
    delete(m, "x")
    fmt.Println(len(m))
}
```

Answer: `0`

Q18

```go
package main
import "fmt"

func main() {
    m := make(map[string]int)
    m["a"]++
    fmt.Println(m["a"])
}
```

Answer: `1` **Explanation:** Missing key reads as zero value.

Q19

```
package main
import "fmt"

func main() {
    s := make([]int, 2, 4)
    fmt.Println(len(s), cap(s))
}
```

Answer: `2 4`

Q20

```
package main
import "fmt"

func main() {
    s := []int{1, 2, 3}
    t := s[:0]
    fmt.Println(len(t), cap(t))
}
```

Answer: `0 3`

Q21

```
package main
import "fmt"

func main() {
    s := []int{1, 2, 3}
    t := append([]int{}, s...)
    t[0] = 100
    fmt.Println(s[0], t[0])
}
```

Answer: `1 100` **Explanation:** `append` to empty literal creates copy.

Q22

```
package main
import "fmt"

func main() {
    var s []int
```

```
    fmt.Println(s == nil, len(s), cap(s))
}
```

Answer: `true 0 0`

Q23

```
package main
import "fmt"

func main() {
    m := map[int]int{1: 10}
    for k := range m {
        delete(m, k)
    }
    fmt.Println(len(m))
}
```

Answer: `0`

Q24

```
package main
import "fmt"

func main() {
    m := map[string]int{"a": 1, "b": 2}
    for k, v := range m {
        fmt.Println(k, v)
    }
}
```

Answer: Two lines, order not guaranteed. **Explanation:** Map iteration order is randomized.

Section 3: Functions, Methods, and Interfaces (Q25-36)

Q25

```
package main
import "fmt"

func add(a, b int) int {
    return a + b
}

func main() {
    fmt.Println(add(2, 3))
}
```

Answer: `5`

Q26

```go
package main
import "fmt"

func f() (x int) {
    x = 1
    defer func() { x++ }()
    return
}

func main() {
    fmt.Println(f())
}
```

Answer: `2` **Explanation:** Deferred closure updates named return value.

Q27

```go
package main
import "fmt"

type C struct{ n int }

func (c C) Inc() { c.n++ }

func main() {
    c := C{1}
    c.Inc()
    fmt.Println(c.n)
}
```

Answer: `1` **Explanation:** Value receiver works on copy.

Q28

```go
package main
import "fmt"

type C struct{ n int }

func (c *C) Inc() { c.n++ }

func main() {
    c := C{1}
    c.Inc()
```

```
    fmt.Println(c.n)
}
```

Answer: `2`

Q29

```
package main
import "fmt"

type I interface{ M() }
type T struct{}

func (T) M() {}

func main() {
    var i I = T{}
    fmt.Println(i != nil)
}
```

Answer: `true`

Q30

```
package main
import "fmt"

type I interface{ M() }
type T struct{}

func (*T) M() {}

func main() {
    var t *T
    var i I = t
    fmt.Println(i == nil)
}
```

Answer: `false` **Explanation:** Interface is non-nil if dynamic type exists.

Q31

```
package main
import "fmt"

func main() {
    var any interface{} = 42
    v, ok := any.(int)
```

```
    fmt.Println(v, ok)
}
```

Answer: `42 true`

Q32

```
package main
import "fmt"

func main() {
    var any interface{} = "go"
    _, ok := any.(int)
    fmt.Println(ok)
}
```

Answer: `false`

Q33

```
package main
import "fmt"

func main() {
    fmt.Printf("%T\n", 3.14)
}
```

Answer: `float64`

Q34

```
package main
import "fmt"

func sum(nums ...int) int {
    s := 0
    for _, n := range nums {
        s += n
    }
    return s
}

func main() {
    fmt.Println(sum(), sum(1, 2, 3))
}
```

Answer: `0 6`

Q35

```go
package main
import "fmt"

func main() {
    a := []int{1, 2, 3}
    fmt.Println(a...)
}
```

Answer: Compile error. **Explanation:** `...` cannot be used in `Println` this way.

Q36

```go
package main
import "fmt"

func main() {
    f := func(x int) int { return x * x }
    fmt.Println(f(4))
}
```

Answer: `16`

Section 4: Concurrency and Channels (Q37-50)

Q37

```go
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 10
    fmt.Println(<-ch)
}
```

Answer: `10`

Q38

```go
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    close(ch)
    v, ok := <-ch
    fmt.Println(v, ok)
}
```

Answer: `0 false`

Q39

```
package main
import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    close(ch)
    for v := range ch {
        fmt.Print(v)
    }
}
```

Answer: `12`

Q40

```
package main
import "fmt"

func main() {
    ch := make(chan struct{})
    go func() {
        fmt.Print("A")
        close(ch)
    }()
    <-ch
    fmt.Print("B")
}
```

Answer: `AB`

Q41

```
package main
import "fmt"

func main() {
    ch := make(chan int)
    go func() { ch <- 7 }()
    fmt.Println(<-ch)
}
```

Answer: `7`

Q42

```go
package main
import "fmt"

func main() {
    ch := make(chan int)
    select {
    case v := <-ch:
        fmt.Println(v)
    default:
        fmt.Println("default")
    }
}
```

Answer: `default`

Q43

```go
package main
import (
    "fmt"
    "time"
)

func main() {
    select {
    case <-time.After(10 * time.Millisecond):
        fmt.Println("timeout")
    }
}
```

Answer: `timeout`

Q44

```go
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 5
    select {
    case v := <-ch:
        fmt.Println(v)
    default:
        fmt.Println("none")
```

```
    }
}
```

Answer: `5`

Q45

```
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 1
    close(ch)
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Answer: `1` then `0`

Q46

```
package main
import "fmt"

func main() {
    defer fmt.Print("1")
    defer fmt.Print("2")
    fmt.Print("3")
}
```

Answer: `321`

Q47

```
package main
import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        go fmt.Println(i)
    }
}
```

Answer: Nondeterministic / may print nothing. **Explanation:** `main` may exit before goroutines run.

Q48

```go
package main
import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(v int) {
            defer wg.Done()
            fmt.Print(v)
        }(i)
    }
    wg.Wait()
}
```

Answer: Prints `0`, `1`, `2` in any order.

Q49

```go
package main
import "fmt"

func main() {
    ch := make(chan int)
    go func() {
        ch <- 1
        ch <- 2
        close(ch)
    }()
    for v := range ch {
        fmt.Print(v)
    }
}
```

Answer: `12`

Q50

```go
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 1
    ch <- 2
```

```
    fmt.Println(<-ch)
}
```

Answer: Deadlock/panic scenario (blocks on second send).

Section 5: Interview Rapid Fire (Q51-60)

Q51

What is output?

```
fmt.Println("go" == "go")
```

Answer: `true`

Q52

```
fmt.Println([]byte("A"))
```

Answer: `[65]`

Q53

```
fmt.Println('A')
```

Answer: `65`

Q54

```
fmt.Println("A"[0])
```

Answer: `65`

Q55

```
fmt.Println(string([]byte{65, 66}))
```

Answer: `AB`

Q56

```
fmt.Println(len("你好"))
```

Answer: `6` **Explanation:** UTF-8 bytes, not rune count.

Q57

```
fmt.Println(len([]rune("你好")))
```

Answer: `2`

Q58

```
var m map[string]int
fmt.Println(m == nil)
```

Answer: `true`

Q59

```
var s []int
fmt.Println(s == nil)
```

Answer: `true`

Q60

```
fmt.Println(make([]int, 0) == nil)
```

Answer: `false`

Section 6: Synchronization, Mutex, RWMutex, and Atomics (Q61-75)

Q61

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex
    x := 0
    mu.Lock()
    x++
    mu.Unlock()
    fmt.Println(x)
}
```

Answer: `1`

Q62

```go
package main
import (
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex
    var wg sync.WaitGroup
    x := 0
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock(); x++; mu.Unlock()
        }()
    }
    wg.Wait()
    fmt.Println(x)
}
```

Answer: `1000`

Q63

```go
package main
import (
    "fmt"
    "sync"
)

func main() {
    var rw sync.RWMutex
    x := 10
    rw.RLock()
    fmt.Println(x)
    rw.RUnlock()
}
```

Answer: `10`

Q64

```go
package main
import (
    "fmt"
    "sync/atomic"
)
```

```
func main() {
    var x int64
    atomic.AddInt64(&x, 2)
    atomic.AddInt64(&x, 3)
    fmt.Println(atomic.LoadInt64(&x))
}
```

Answer: `5`

Q65

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var once sync.Once
    n := 0
    f := func() { n++ }
    once.Do(f)
    once.Do(f)
    fmt.Println(n)
}
```

Answer: `1`

Q66

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    go func() {
        defer wg.Done()
        fmt.Print("A")
    }()
    wg.Wait()
    fmt.Print("B")
}
```

Answer: `AB`

Q67

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var m sync.Map
    m.Store("x", 1)
    v, _ := m.Load("x")
    fmt.Println(v)
}
```

Answer: `1`

Q68

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var m sync.Map
    actual, loaded := m.LoadOrStore("k", 10)
    fmt.Println(actual, loaded)
    actual, loaded = m.LoadOrStore("k", 20)
    fmt.Println(actual, loaded)
}
```

Answer: `10 false` then `10 true`

Q69

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    var cond = sync.NewCond(&sync.Mutex{})
    ready := false
    go func() {
        cond.L.Lock()
        ready = true
```

```
        cond.Signal()
        cond.L.Unlock()
    }()
    cond.L.Lock()
    for !ready { cond.Wait() }
    cond.L.Unlock()
    fmt.Println("ok")
}
```

Answer: `ok`

Q70

```
package main
import (
    "fmt"
    "sync"
)

func main() {
    pool := sync.Pool{New: func() any { return 42 }}
    fmt.Println(pool.Get())
}
```

Answer: `42`

Q71

```
package main
import "fmt"

func main() {
    ch := make(chan int)
    go func() { close(ch) }()
    _, ok := <-ch
    fmt.Println(ok)
}
```

Answer: `false`

Q72

```
package main
import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
```

```
    fmt.Println(len(ch), cap(ch))
}
```

Answer: `2 2`

Q73

```
package main
import "fmt"

func main() {
    ch := make(chan int)
    go func() {
        ch <- 1
        close(ch)
    }()
    for v := range ch { fmt.Print(v) }
}
```

Answer: `1`

Q74

```
package main
import (
    "context"
    "fmt"
)

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    cancel()
    select {
    case <-ctx.Done():
        fmt.Println(ctx.Err())
    }
}
```

Answer: `context canceled`

Q75

```
package main
import "fmt"

func main() {
    defer fmt.Print("A")
    defer fmt.Print("B")
```

```
    defer fmt.Print("C")
}
```

Answer: `CBA`

Section 7: Sorting, Collections-Like Behavior, Interfaces, and Generics (Q76-95)

Q76

```
package main
import (
    "fmt"
    "sort"
)

func main() {
    a := []int{3,1,2}
    sort.Ints(a)
    fmt.Println(a)
}
```

Answer: `[1 2 3]`

Q77

```
package main
import (
    "fmt"
    "sort"
)

func main() {
    a := []string{"go", "java", "py"}
    sort.Slice(a, func(i, j int) bool { return len(a[i]) < len(a[j]) })
    fmt.Println(a)
}
```

Answer: `[go py java]`

Q78

```
package main
import "fmt"

func main() {
    a := []int{1,2,3}
    b := a[:]
```

```
    b[1] = 99
    fmt.Println(a)
}
```

Answer: `[1 99 3]`

Q79

```
package main
import "fmt"

func main() {
    a := []int{1,2,3}
    b := append(a[:0:0], a...)
    b[0] = 7
    fmt.Println(a, b)
}
```

Answer: `[1 2 3] [7 2 3]`

Q80

```
package main
import "fmt"

func main() {
    arr := [5]int{1,2,3,4,5}
    s := arr[1:3]
    fmt.Println(len(s), cap(s))
}
```

Answer: `2 4`

Q81

```
package main
import "fmt"

func main() {
    m := map[string]int{"a":1}
    if v, ok := m["a"]; ok { fmt.Println(v) }
}
```

Answer: `1`

Q82

```go
package main
import "fmt"

type S struct { x int }

func (s S) F() { fmt.Print("V") }
func (s *S) G() { fmt.Print("P") }

func main() {
    v := S{1}
    v.F()
    v.G()
}
```

Answer: `VP`

Q83

```go
package main
import "fmt"

type I interface { G() }
type S struct{}
func (s *S) G() {}

func main() {
    var i I
    var s S
    _ = s
    fmt.Println(i == nil)
}
```

Answer: `true`

Q84

```go
package main
import "fmt"

func min[T ~int | ~float64](a, b T) T {
    if a < b { return a }
    return b
}

func main() {
    fmt.Println(min(3, 2), min(3.5, 7.1))
}
```

Answer: `2 3.5`

Q85

```
package main
import "fmt"

func main() {
    var any any = "go"
    s, ok := any.(string)
    fmt.Println(s, ok)
}
```

Answer: `go true`

Q86

```
package main
import "fmt"

func main() {
    var any any = 10
    s, ok := any.(string)
    fmt.Println(s, ok)
}
```

Answer: `false` **Explanation:** Zero value for string is empty, `ok` false.

Q87

```
package main
import "fmt"

func main() {
    var i interface{} = (*int)(nil)
    fmt.Println(i == nil)
}
```

Answer: `false`

Q88

```
package main
import "fmt"

func main() {
    var i interface{} = nil
    fmt.Println(i == nil)
}
```

Answer: `true`

Q89

```
package main
import "fmt"

func main() {
    fmt.Println("go" < "java")
}
```

Answer: `false`

Q90

```
package main
import "fmt"

func main() {
    b := []byte("AB")
    b[0] = 'Z'
    fmt.Println(string(b))
}
```

Answer: `ZB`

Q91

```
package main
import "fmt"

func main() {
    r := []rune("你好")
    r[0] = '您'
    fmt.Println(string(r))
}
```

Answer: 您好

Q92

```
package main
import "fmt"

func main() {
    for i, r := range "go" {
        fmt.Print(i, ":", string(r), " ")
```

```
    }
}
```

Answer: `0:g 1:o`

Q93

```
package main
import "fmt"

func main() {
    a := map[int]int{}
    for i := 0; i < 3; i++ { a[i] = i*i }
    fmt.Println(len(a))
}
```

Answer: `3`

Q94

```
package main
import "fmt"

func main() {
    a := []int{1,2,3,4}
    a = append(a[:1], a[2:]...)
    fmt.Println(a)
}
```

Answer: `[1 3 4]`

Q95

```
package main
import "fmt"

func main() {
    var z int
    fmt.Printf("%T %v\n", z, z)
}
```

Answer: `int 0`

Section 8: Cache and Production Patterns in Go (Q96-120)

Q96

```go
package main
import "fmt"

func main() {
    cache := map[string]int{}
    if _, ok := cache["x"]; !ok { cache["x"] = 1 }
    if _, ok := cache["x"]; !ok { cache["x"] = 2 }
    fmt.Println(cache["x"])
}
```

Answer: `1`

Q97

```go
package main
import "fmt"

type entry struct { val int; ok bool }

func main() {
    cache := map[string]entry{}
    cache["a"] = entry{0, true}
    v := cache["a"]
    fmt.Println(v.val, v.ok)
}
```

Answer: `0 true`

Q98

```go
package main
import (
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex
    cache := map[string]int{}
    mu.Lock(); cache["k"] = 10; mu.Unlock()
    mu.Lock(); fmt.Println(cache["k"]); mu.Unlock()
}
```

Answer: `10`

Q99

```go
package main
import (
```

```
    "fmt"
    "sync"
)

func main() {
    var m sync.Map
    m.Store("a", 1)
    m.Range(func(k, v any) bool {
        fmt.Print(k, v)
        return true
    })
}
```

Answer: `a1`

Q100

```
package main
import (
    "fmt"
    "time"
)

func main() {
    ttl := map[string]time.Time{"k": time.Now().Add(-time.Second)}
    _, alive := ttl["k"]
    fmt.Println(alive, time.Now().Before(ttl["k"]))
}
```

Answer: `true false`

Q101

```
package main
import "fmt"

func main() {
    store := map[string]int{"a": 1}
    v := store["missing"]
    fmt.Println(v)
}
```

Answer: `0`

Q102

```
package main
import "fmt"
```

```go
func main() {
    type item struct { hits int }
    c := map[string]*item{}
    c["x"] = &item{}
    c["x"].hits++
    fmt.Println(c["x"].hits)
}
```

Answer: `1`

Q103

```go
package main
import (
    "fmt"
    "time"
)

func main() {
    start := time.Now()
    time.Sleep(10 * time.Millisecond)
    fmt.Println(time.Since(start) >= 10*time.Millisecond)
}
```

Answer: `true`

Q104

```go
package main
import "fmt"

func main() {
    type key struct { id int }
    m := map[key]int{{1}: 10}
    fmt.Println(m[key{1}])
}
```

Answer: `10`

Q105

```go
package main
import "fmt"

func main() {
    type key struct { id []int }
    _ = key{}
    fmt.Println("compile error if used as map key")
}
```

Answer: Slices in struct make it non-comparable, cannot be map key.

Q106

```
package main
import "fmt"

func main() {
    cache := map[string]int{"a": 1}
    delete(cache, "a")
    fmt.Println(len(cache))
}
```

Answer: `0`

Q107

```
package main
import "fmt"

func main() {
    m := map[string]int{"a": 1}
    for k := range m { m[k]++ }
    fmt.Println(m["a"])
}
```

Answer: `2`

Q108

```
package main
import "fmt"

func main() {
    type cache map[string]int
    inc := func(c cache, k string) { c[k]++ }
    c := cache{"x": 0}
    inc(c, "x")
    fmt.Println(c["x"])
}
```

Answer: `1`

Q109

```
package main
import "fmt"

func main() {
```

```
    ids := []int{1, 2, 3}
    set := map[int]struct{}{}
    for _, id := range ids { set[id] = struct{}{} }
    _, ok := set[2]
    fmt.Println(ok)
}
```

Answer: `true`

Q110

```
package main
import "fmt"

func main() {
    const miss = -1
    cache := map[string]int{}
    v, ok := cache["x"]
    if !ok { v = miss }
    fmt.Println(v)
}
```

Answer: `-1`

Q111

```
package main
import "fmt"

func main() {
    var p *int = nil
    fmt.Println(p == nil)
}
```

Answer: `true`

Q112

```
package main
import "fmt"

func main() {
    err := fmt.Errorf("cache miss")
    fmt.Println(err != nil)
}
```

Answer: `true`

Q113

```go
package main
import "fmt"

func main() {
    cache := map[string]int{"a": 1}
    if v, ok := cache["a"]; ok {
        fmt.Println(v)
    }
}
```

Answer: `1`

Q114

```go
package main
import "fmt"

func main() {
    c := make(chan int, 1)
    select {
    case c <- 1:
        fmt.Println("sent")
    default:
        fmt.Println("drop")
    }
}
```

Answer: `sent`

Q115

```go
package main
import "fmt"

func main() {
    c := make(chan int, 1)
    c <- 1
    select {
    case c <- 2:
        fmt.Println("sent")
    default:
        fmt.Println("drop")
    }
}
```

Answer: `drop`

Q116

```go
package main
import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 7
    close(ch)
    for v := range ch { fmt.Println(v) }
}
```

Answer: `7`

Q117

```go
package main
import "fmt"

func main() {
    f := func() (x int) {
        x = 1
        defer func() { x = 5 }()
        return
    }
    fmt.Println(f())
}
```

Answer: `5`

Q118

```go
package main
import "fmt"

func main() {
    var s []int
    fmt.Println(append(s, 1, 2))
}
```

Answer: `[1 2]`

Q119

```go
package main
import "fmt"

func main() {
    a := make([]int, 0, 1)
    b := append(a, 1)
    c := append(b, 2)
```

```
    fmt.Println(a, b, c)
}
```

Answer: `[] [1] [1 2]`

Q120

```
package main
import "fmt"

func main() {
    fmt.Println("Production note: protect shared caches with mutex/sync.Map and validate
determinism in tests.")
}
```

Answer: Prints the exact line.

Interview Tips

- State if output is deterministic or order-dependent.
- Mention nil interface vs typed nil pointer inside interface.
- Mention map iteration is unordered by language design.
- For concurrency snippets, always discuss scheduling and main goroutine exit.
- For shared counters/caches, call out atomic vs mutex trade-offs.
- Mention slice aliasing and backing-array reuse whenever append/slicing appears.
- In channel questions, distinguish closed-channel receive from blocked send/receive.
- For production scenarios, mention race detector (`go test -race`) as a validation tool.

Companion: Regular Problems + Solutions (One-Stop)

For non-tricky, implementation-focused interview prep (slices/maps, channels, goroutines, caching, and DSA/Algo), use:

- `40_GO_ONE_STOP_DSA_ALGO_REGULAR_PROBLEMS.md`